$$
\begin{array}{rl}
e ::= x \mid c \mid \text{true} \mid \text{false} \mid k_{\#n} \mid () & \text{constants} \\
\quad \mid \textbf{let}\ x = e_1\ \textbf{in}\ e_2 & \text{let binding} \\
\quad \mid \textbf{uniform}(e_1, e_2) \mid \textbf{gaussian}(e_1, e_2) \mid \textbf{exponential}(e) \mid \ldots & \text{continuous distributions} \\
\quad \mid \textbf{discrete}(p_0, \ldots, p_{n-1}) & \text{discrete distribution} \\
\quad \mid e_1 < e_2 \mid e_1 \le e_2 \mid e_1 <_{\#n} e_2 \mid e_1 \le_{\#n} e_2 & \text{comparisons} \\
\quad \mid e_1 \,\&\&\, e_2 \mid e_1 \,||\, e_2 \mid \text{not } e & \text{Boolean operations} \\
\quad \mid \textbf{if}\ e_1\ \textbf{then}\ e_2\ \textbf{else}\ e_3 & \text{conditional} \\
\quad \mid (e_1, e_2) \mid \textbf{fst}\ e \mid \textbf{snd}\ e & \text{pairs} \\
\quad \mid \textbf{fun}\ x\ \to\ e \mid e_1\ e_2 \mid \textbf{fix}\ f\ x := e & \text{functions} \\
\quad \mid \textbf{observe}\ e \mid e_1; e_2 & \text{effects} \\
\quad \mid \text{nil} \mid e_1 :: e_2 \mid (\textbf{match}\ e\ \textbf{with} \mid \text{nil} \to e_{\text{nil}} \mid e_1 :: e_2 \to e_{\text{cons}}\ \textbf{end}) & \text{lists} \\
\quad \mid \textbf{diverge} & \text{diverge}
\end{array}
$$

Fig. 5. Syntax of the Slice language. Slice supports a variety of continuous distributions with one and two parameters; for a full list of supported distributions, see Appendix C.

We now turn to the more involved example in Figure 4, which Slice discretizes automatically. This example emphasizes how our discretization procedure operates across higher-order functions, unbounded recursion, and lists.

The program models the following process: We start by randomly sampling 4 intervals (l. 6 and 13), stored as a list of pairs, where the first component is a flag and the second component is a function that determines whether a given point falls within that interval. We then employ unbounded recursion (l. 36) to repeat the following a potentially unbounded number of times according to a geometric distribution: sample a point y from a gaussian (l. 31), and update the flag of every interval to true whenever y falls within that interval (l. 22). The program returns true iff *all* intervals have been hit at least once (l. 50 and 42). Exact inference on this program therefore determines the probability that this infinite stochastic process hits every interval at least once.

Slice *automatically* produces a *provably equivalent* (cf. §6) discrete program (shown in §A.5). Even though this program still contains unbounded recursion, our back-end automatically determines that its stochastic execution behavior can be modeled as a *finite-state* Markov chain. Our key insight is that exact inference can then be performed via *probabilistic model checking*: we use Storm [16], which automatically computes that the sought-after probability is 0.034474.... See §7.1 for details. To the best of our knowledge, our pipeline is the first capable of performing exact inference on suitable programs containing continuous sampling, higher-order functions, and unbounded recursion. In §7.1.2, we extend this program by conditioning *inside the recursion*.

3 The Slice Language and Type System

We now describe the language (§3.1), its types (§3.2), and the typing rules for discretization (§3.3). Henceforth, we use the terms *expression*, *subexpression*, and *program* somewhat interchangeably when the intended meaning is clear from context.

3.1 The Slice Language

The syntax in Figure 5 extends a standard higher-order functional core with probabilistic choices. Beyond constants, variables, comparisons, Boolean operators, conditionals, pairs, and lists, Slice offers first-class functions, recursion, and observations. The construct **observe**(e) conditions on e holding. Finite-domain constants $k_{\#n}$ range over $\{0_{\#n}, \ldots, (n-1)_{\#n}\}$ as discussed below. For random sampling, we include the usual discrete distributions and any continuous distribution with a tractable cumulative distribution function (one- or two-argument variants such as exponential, laplace, uniform, gaussian, beta, gamma). Adding further distributions is straightforward as long as their CDFs are known; §C lists the ones implemented.

The expression **discrete**$(p_0, \ldots, p_{n-1})$ requires a nonempty probability vector: $n \ge 1$, every $p_i \ge 0$, and $\sum_i p_i = 1$. It returns $k_{\#n}$ with probability p_k.

These are ideal real-valued premises. The implementation separately validates finite binary64 weights. It normalizes compiler-generated weights only within a tight allowance for accumulated CDF arithmetic, and parses source vectors within a documented 10^{-4} rounded-input allowance. These engineering policies do not replace the exact premise above: binary64 evaluation and normalization do not by themselves establish exact preservation of ideal real-valued probabilities, which requires a separate numerical error analysis.

3.2 Types

We formalize the types that Slice supports. A *value* set V is a finite set of reals or $\top$. Elements of $\{{<}x \mid x \in \mathbb{R}\} \cup \{{\leq}x \mid x \in \mathbb{R}\}$ are called *cuts*. Cut sets induce intervals partitioning the reals as follows: Given a We order cuts first by their real boundary and then by their tag, with ${<}c \prec {\leq}c$ at a shared boundary c. Thus every finite non-empty cut set has a unique enumeration $B = \{{\sim_1}x_1, \ldots, {\sim_n}x_n\}$ with $x_1 \leq \ldots \leq x_n$, we define the set Intervals(B) of intervals it induces as expected, e.g., in increasing tagged-cut order. For a cut $b = {\sim}x$, write $S_b = \{r \in \mathbb{R} \mid r \sim x\}$. The canonical order makes $S_{b_1} \subsetneq \cdots \subsetneq S_{b_n}$. We define the indexed partition induced by B by

$$\text{Intervals}(B)_0 = S_{b_1}, \qquad \text{Intervals}(B)_i = S_{b_{i+1}} \setminus S_{b_i} \ (0 < i < n), \qquad \text{Intervals}(B)_n = \mathbb{R} \setminus S_{b_n}.$$

In particular, Intervals($\{{<}0.1, {\leq}0.5\}$) $= \{(-\infty, 0.1), [0.1, 0.5], (0.5, \infty)\}$. The strict and non-strict cuts differ only in which side contains the boundary point: Intervals($\{{<}c\}$) $= \{(-\infty, c), [c, \infty)\}$, whereas Intervals($\{{\leq} c\}$) $= \{(-\infty, c], (c, \infty)\}$. Thus a boundary generated by $e < c$ places c on the right, while a boundary generated by $e \leq c$ places c on the left. Moreover, we let Intervals($\emptyset$) $= \{(-\infty, \infty)\}$ and Intervals($\top$) $= 2^{\mathbb{R}}$. We denote by Intervals(B)$_i$ the i-th interval of Intervals(B), assuming Intervals(B) is 0-indexed, i.e., $i \in \{0, ..., n\}$Intervals($\emptyset$)$_0 = \mathbb{R}$; Intervals($\top$) is not used because $\top$ denotes an undiscretized representation. The construction above consists of $n + 1$ pairwise disjoint intervals whose union is $\mathbb{R}$, so every real has a unique interval index. We denote the set of all cut- and value sets, respectively, by $\mathcal{B}$ and $\mathcal{V}$.

Definition 3.1 (Types). For $n \in \mathbb{N}$, $B \in \mathcal{B}$, and $V \in \mathcal{V}$, types τ adhere to the grammar

$$\tau ::= \textbf{int} \mid \textbf{bool} \mid \textbf{unit} \mid \textbf{fin}(n) \mid \textbf{float}[B; V] \mid \tau_1 * \tau_2 \mid \tau_1 \to \tau_2 \mid \textbf{list}(\tau)\ .$$

Standard types cover integers, booleans, ordered pairs, functions, and lists. Finite types $\textbf{fin}(n)$ contain $\{0_{\#n}, \ldots, (n-1)_{\#n}\}$; the $\#n$-subscript reminds inference backends such as Dice that these values live in a finite domain. The float type $\textbf{float}[B; V]$ annotates a real-valued expression with its cut set B (an over-approximation of the comparisons it participates in) and value set V (an over-approximation of the values it can take). $B = \top$ means the expression may be compared to arbitrary reals; $B = \emptyset$ means it will never be compared; $V = \top$ indicates an unrestricted result.

3.3 Typing Rules

The typing rules are split into three parts: Rules specific to float types and continuous distributions (Figure 6), standard rules for all other language constructs (Figure 7), and subtyping (Figure 8).

To understand the float typing rules, it is helpful to understand how discretization works. A type **float**[B; V] is discretized as follows: If $B \neq \top$ is a finite set of cuts, then the type is discretized to a type $\textbf{fin}(n+1)$ where $n + 1 = |\text{Intervals}(B)|$ is the number of intervals induced by B. For instance, if $B = \{{<}0.5, {\leq}0.8\}$, then the type is discretized to $\textbf{fin}(3)$, where $0_{\#3}$ represents the interval $(-\infty, 0.5)$, $1_{\#3}$ represents the interval $[0.5, 0.8]$, and $2_{\#3}$ represents the interval $(0.8, \infty)$. If $B = \top$, then no discretization is performed, and the type remains **float**.

To understand the typing rules, keep in mind that the cut sets B need to be sufficiently fine-grained — i.e., contain enough cuts — so that the discretized program has enough information to determine how the original continuous program behaved.

Constants. Rule Float-Const assigns a constant c the type $\textbf{float}[B; V]$ whenever has(V, c) holds (i.e., $c \in V$ or $V = \top$). Constants do not constrain the cut set, but recording them in V ensures later comparisons know which values must remain distinguishable.

$$\textsc{Var}\ \frac{}{\Gamma, x : \tau \vdash x : \tau} \qquad \textsc{Let}\ \frac{\Gamma \vdash e_1 : \tau_1 \qquad \Gamma, x : \tau_1 \vdash e_2 : \tau_2}{\Gamma \vdash \mathbf{let}\ x = e_1\ \mathbf{in}\ e_2 : \tau_2} \qquad \textsc{If}\ \frac{\Gamma \vdash e_1 : \mathbf{bool} \qquad \Gamma \vdash e_2 : \tau \qquad \Gamma \vdash e_3 : \tau}{\Gamma \vdash \mathbf{if}\ e_1\ \mathbf{then}\ e_2\ \mathbf{else}\ e_3 : \tau}$$

$$\textsc{Observe}\ \frac{\Gamma \vdash e : \mathbf{bool}}{\Gamma \vdash \mathbf{observe}\ e : \mathbf{unit}} \qquad \textsc{Discrete}\ \frac{n \geq 1 \qquad \forall i \in \{0, \ldots, n-1\}.\ p_i \geq 0 \qquad \sum_{i=0}^{n-1} p_i = 1}{\Gamma \vdash \mathbf{discrete}(p_0, \ldots, p_{n-1}) : \mathbf{fin}(n)} \qquad \textsc{FinConst}\ \frac{0 \leq k < n}{\Gamma \vdash k_{\#n} : \mathbf{fin}(n)}$$

$$\textsc{FinLess}\ \frac{\Gamma \vdash e_1 : \mathbf{fin}(n) \qquad \Gamma \vdash e_2 : \mathbf{fin}(n)}{\Gamma \vdash e_1 <_{\#n} e_2 : \mathbf{bool}} \qquad \textsc{FinLessEq}\ \frac{\Gamma \vdash e_1 : \mathbf{fin}(n) \qquad \Gamma \vdash e_2 : \mathbf{fin}(n)}{\Gamma \vdash e_1 \leq_{\#n} e_2 : \mathbf{bool}} \qquad \textsc{Diverge}\ \frac{}{\Gamma \vdash \mathbf{diverge} : \tau}$$

Fig. 7. Typing rules for general language constructs

$$\textsc{Sub-Refl}\ \frac{\tau \in \{\mathbf{int}, \mathbf{bool}, \mathbf{fin}(n)\}}{\tau <: \tau} \qquad \textsc{Sub-Float}\ \frac{B_1 = B_2 \qquad V_1 \sqsubseteq V_2}{\mathbf{float}[B_1; V_1] <: \mathbf{float}[B_2; V_2]} \qquad \textsc{Sub-Pair}\ \frac{\tau_1 <: \tau_1' \qquad \tau_2 <: \tau_2'}{\tau_1 * \tau_2 <: \tau_1' * \tau_2'}$$

$$\textsc{Sub-List}\ \frac{\tau <: \tau'}{\mathbf{list}(\tau) <: \mathbf{list}(\tau')} \qquad \textsc{Sub-Arrow}\ \frac{\tau_1' <: \tau_1 \qquad \tau_2 <: \tau_2'}{\tau_1 \to \tau_2 <: \tau_1' \to \tau_2'} \qquad \textsc{Sub-Type}\ \frac{\tau_1 <: \tau_2 \qquad \Gamma \vdash e : \tau_1}{\Gamma \vdash e : \tau_2}$$

Fig. 8. Subtyping relation. For both cut- and value sets, we let $d \sqsubseteq d'$ iff $\begin{cases} d \subseteq d' & \text{if } d \neq \top \text{ and } d' \neq \top \\ d' = \top & \text{otherwise .} \end{cases}$

(3) If $B \neq \top$ and $V_1 \neq \top$, then the situation is similar to the previous case.

Note that $B = \top$ iff $V = \top$ accounts for comparisons where a continuously ranging expression is compared against arbitrary real values not statically known, such as other continuous distributions, in which case no discretization can be performed. Additionally, we insist that $B_1 = B_2$, which ensures that both comparands will be discretized to the same type, or not discretized at all.

Finally, for non-strict comparisons, the rule is analogous to Definition 3.3:

Definition 3.4. Let $B \in \mathcal{B}$ and $V_1, V_2 \in \mathcal{V}$. We say that $\text{answers}_{\leq}(B, V_1, V_2)$ if (1) $B = \top$, or (2) $B \neq \top$ and $V_2 \neq \top$ and $\forall x \in V_2,\ \leq x \in B$, or (3) $B \neq \top$ and $V_1 \neq \top$ and $\forall x \in V_1,\ < x \in B$.

The tagged-cut convention is essential here. If both $<c$ and $\leq c$ occur, their order isolates $\{c\}$ between them. More generally, membership in each predicate set S_b is determined by the interval index. This does not make a comparison constant on every arbitrary pair of induced intervals. Instead, the typing premises give the property needed on reachable values. For $\text{answers}_{<}(B, V_1, V_2)$, either the right operand is known to lie in V_2 and $< y \in B$ for every reachable $y \in V_2$, or the left operand is known to lie in V_1 and $\leq x \in B$ for every reachable $x \in V_1$. In the first case the interval index decides $x < y$ for every reachable pair, and in the second case it decides the same predicate symmetrically. Replacing $< y$ and $\leq x$ by the cuts in $\text{answers}_{\leq}(B, V_1, V_2)$ gives the corresponding reachable-pair statement for $x \leq y$.

Subtyping. The type system includes a subtyping relation that captures when one type can be replaced by another. The subtyping rules are shown in Figure 8. For float types $\mathbf{float}[B; V]$, subtyping follows the lattice ordering on V: an expression known to range over some set of values can also be considered to range over a superset of those values. For B, we require equality to make sure that the discretized representation of the expression is not changed by subtyping.

All other typing rules are standard and depicted in Figure 7 and §D, respectively.

4 Type Inference

We now describe our type inference algorithm, whose goal is to annotate each subexpression of a program with its most precise type. Specifically, we focus on expressions of type $\mathbf{float}[B; V]$ and show how Slice populates the cut set B and value set V, which later determine how and whether the expression can be discretized. Type inference for all other types is standard and follows the classic Hindley-Milner algorithm. For float-typed

Distribution	Density	Distribution	Density
Exponential	$\lambda e^{-\lambda x}$	Uniform	$\frac{1}{b-a}\mathbf{1}_{[a,b]}(x),\ a<b$
Laplace	$\frac{1}{2b}e^{-\frac{\lvert x-\mu\rvert}{b}}$	Gaussian	$\frac{1}{\sqrt{2\pi}\sigma}e^{-\frac{(x-\mu)^2}{2\sigma^2}}$
Cauchy	$\frac{1}{\pi\gamma\left[1+\left(\frac{x-x_0}{\gamma}\right)^2\right]}$	Beta	$\frac{x^{\alpha-1}(1-x)^{\beta-1}}{B(\alpha,\beta)}$
Student-T	$c\left(1+\frac{x^2}{\nu}\right)^{-\frac{\nu+1}{2}}$	LogNormal	$\frac{1}{x\sigma\sqrt{2\pi}}e^{-\frac{(\ln x-\mu)^2}{2\sigma^2}}$
Chi-squared	$\frac{1}{2^{k/2}\Gamma(k/2)}x^{k/2-1}e^{-x/2}$	Gamma	$\frac{\beta^\alpha}{\Gamma(\alpha)}x^{\alpha-1}e^{-\beta x}$
Logistic	$\frac{e^{-(x-\mu)/s}}{s(1+e^{-(x-\mu)/s})^2}$	Pareto	$\frac{\alpha x_m^\alpha}{x^{\alpha+1}}$
Rayleigh	$\frac{x}{\sigma^2}e^{-x^2/(2\sigma^2)}$	Weibull	$\frac{k}{\lambda}\left(\frac{x}{\lambda}\right)^{k-1}e^{-(x/\lambda)^k}$
		Gumbel-1	$\frac{1}{\beta}e^{-(z+e^{-z})},\quad z=\frac{x-\mu}{\beta}$
		Gumbel-2	$\frac{\alpha\beta x^{-\alpha-1}e^{-(x/\beta)^{-\alpha}}}{\beta}$
		Exponentiated Power	$k\lambda x^{k-1}e^{-(x/\lambda)^k}$
Discrete Distributions			
Poisson	$\frac{\lambda^k e^{-\lambda}}{k!}$	Geometric	$(1-p)^{k-1}p$
Binomial	$\binom{n}{k}p^k(1-p)^{n-k}$	Negative Binomial	$\binom{k+r-1}{k}(1-p)^r p^k$
Hypergeometric	$\frac{\binom{K}{k}\binom{N-K}{n-k}}{\binom{N}{n}}$	Bernoulli	$p^k(1-p)^{1-k}$
Discrete Uniform	$\frac{1}{n}$	Multinomial	$\frac{n!}{x_1!\cdots x_k!}p_1^{x_1}\cdots p_k^{x_k}$

- $e_1, e_2 \in \mathbb{R}$ and $e_1 \geq e_2$: then $\mathcal{D}[\![\texttt{uniform}(\ldots) : \texttt{float}[B;\top]]\!] = \texttt{diverge}$, which is well-typed at any type by Diverge.
- $B = \top$ and the literal-invalid condition above does not hold: then $\mathcal{D}[\![\texttt{float}[B;\top]]\!] = \texttt{float}$, and

  $$\mathcal{D}[\![\texttt{uniform}(e_1,e_2) : \texttt{float}[B;\top]]\!] = \texttt{uniform}\big(\mathcal{D}[\![e_1 : \texttt{float}[B_1;V_1]]\!],\ \mathcal{D}[\![e_2 : \texttt{float}[B_2;V_2]]\!]\big).$$

  Since recovers(B_1, V_1) and recovers(B_2, V_2) are preserved under discretization, the result is well-typed at $\texttt{float}$ by Continuous-2.
- $B \neq \top$, $e_1, e_2 \in \mathbb{R}$, and $e_1 < e_2$: then $\mathcal{D}[\![\texttt{float}[B;\top]]\!] = \texttt{fin}(|B|+1)$, and

  $$\mathcal{D}[\![\texttt{uniform}(e_1,e_2) : \texttt{float}[B;\top]]\!] = \texttt{discrete}(p_0,\ldots,p_{|B|})$$

  where $p_k = \Pr[X \in \text{Intervals}(B)_k]$ for $X \sim \text{Uniform}(e_1, e_2)$. Hence the distribution is atomless, each $p_k \in [0,1]$, and $\sum_k p_k = 1$, so this $p_k = H(b_{k+1}) - H(b_k)$ for the tagged cumulative function $H(\sim c) = \Pr[X \sim c]$, with artificial outer values 0 and 1. This sub-case has $e_1 < e_2$ because the preceding rule handles equality and reversed bounds. By the canonical tagged-cut order, H is nondecreasing; hence each $p_k = \Pr[X \in \text{Intervals}(B)_k] \in [0,1]$ and the differences telescope to 1. Thus the result is well-typed at $\texttt{fin}(|B|+1)$ by Discrete.
- $B \neq \top$, $B_1 \neq \top$, $B_2 \neq \top$, and at least one of e_1, e_2 is nonliteral: the algorithm generates conditionals over all pairs (v_i, w_j), each branch dispatching to a constant parameter in $\texttt{uniform}$. By the previous sub-case, each valid branch $\mathcal{D}[\![\texttt{uniform}(v_i,w_j) : \texttt{float}[B;\top]]\!]$ is well-typed at $\texttt{fin}(|B|+1) = \mathcal{D}[\![\texttt{float}[B;\top]]\!]$, while equality or reversed bounds use the malformed branch. The conditional structure is well-typed by If (as in the $\texttt{if}$ case above), so the full expression is well-typed at $\mathcal{D}[\![\texttt{float}[B;\top]]\!]$.

Case e is any other language construct (functions, pairs, lists, *etc.*). By definition of $\mathcal{D}[\![\cdot]\!]$, the discretization recurses structurally over e and τ. That is, $\mathcal{D}[\![e:\tau]\!]$ is formed by applying $\mathcal{D}[\![\cdot]\!]$ to each subexpression and subtype of $e:\tau$, and $\mathcal{D}[\![\tau]\!]$ is formed by applying $\mathcal{D}[\![\cdot]\!]$ to each component type of τ. The result then follows immediately by applying the inductive hypothesis to each subterm, analogous to the above cases.

(1) Probability measure. We show for each $x \in X$, the map

$$A \mapsto f(x)(A)$$

is a probability measure.

Fix any $x \in X$. Then $f(x)$ is the zero measure: it satisfies σ-additivity since

$$f(x)\left(\bigcup_i A_i\right) = 0 = \sum_i 0 = \sum_i f(x)(A_i)$$

for every pairwise disjoint family $(A_i)_i$, and total mass $f(x)(Y) = 0 \leq 1$.

(2) Measurable function. For any $A \in \mathcal{F}_Y$, the map $x \mapsto f(x)(A) = 0$ is the constant-zero function on X, hence measurable.

Thus both conditions hold, so f is a Markov kernel. □

Lemma F.33 (Composition of Markov kernels). *Let (X, Σ_X), (Y, Σ_Y), and (Z, Σ_Z) be measurable spaces. If*

$$P : X \to \mathcal{D}(Y) \quad \text{and} \quad Q : Y \to \mathcal{D}(Z)$$

are Markov kernels, then their composition

$$(P \gg\!= Q)(A) \;:=\; \int_{x \in supp(P)} Q(x)(A)\, P(dx)$$

defines a Markov kernel

$$P \gg\!= Q : X \to \mathcal{D}(Z).$$

Proof. See [30, Lemma 1.38]. □

F.4 Small-Step Semantics

Throughout this section, we denote by $e : \tau$ a well-typed expression of type τ and implicitly assume that all sub-expressions are annotated with types, as well. Moreover, for the typed expression $e : \tau = \mathbf{uniform}(e_1 : \mathbf{float}[B_1; V_1], e_2 : \mathbf{float}[B_2; V_2]) : \mathbf{float}[B, V]$ with $B, B_1, B_2 \neq \top$ and $V_1, V_2 \neq \top$, we introduce an auxiliary statement $\mathtt{DC}_{B,B_1,B_2,V_1,V_2}$ and let

$$\mathcal{D}[\![e : \tau]\!] = \mathtt{DC}_{B,B_1,B_2,V_1,V_2}(\mathcal{D}[\![e_1 : \mathbf{float}[B_1; V_1]]\!], \mathcal{D}[\![e_2 : \mathbf{float}[B_2; V_2]]\!])\,.$$

We define the semantics of $\mathtt{DC}_{B,B_1,B_2,V_1,V_2}$ in such a way that it behaves like the discretized program in the last case of Figure 13 but resolves the let-bindings and the conditional choices in a single step, which will simplify our proof. We equip this statement with the following typing rule :

$$\frac{e_1 : \mathbf{fin}(|B_1| + 1) \qquad e_2 : \mathbf{fin}(|B_2| + 1) \qquad \mathsf{recovers}(B_1, V_1) \qquad \mathsf{recovers}(B_2, V_2)}{\mathtt{DC}_{B,B_1,B_2,V_1,V_2}(e_1, e_2) : \mathbf{fin}(|B| + 1)}\ \textsc{TDiscreteCase}$$

Equipped with these notions, we define the small-step semantics of our language:

Definition F.34. The small-step semantics $[\![e : \tau]\!] \in \mathcal{D}(Expr_\tau + \bot)$ is defined recursively on the structure of $e : \tau$ by the rules in Table 1 and Table 2.

In the Discrete row, the typing premises ensure that the displayed weights form a nonempty probability vector. Hence $\mathrm{Discrete}(p_0, \ldots, p_{n-1}) = \sum_{i=0}^{n-1} p_i\, \delta_{i \# n}$ is a probability measure in $\mathcal{D}(Expr_{\mathbf{fin}(n)} + \bot)$. The missing probability mass in $[\![e : \tau]\!]$ is the probability of encountering an error due to ill-formed parameters fed into distributions (e.g., $\mathbf{uniform}(0.8, 0.1)$). $[\![e : \tau]\!](\bot)$ is the probability of encountering an observation failure.

Lemma F.35 (Measurability of constant extension map). *Let $A \subseteq \mathbb{R}^n$ be equipped with the subspace σ-algebra*

$$\mathcal{F}_A := \{A \cap M \mid M \in \mathcal{B}(\mathbb{R}^n)\},$$

and let $B \subseteq \mathbb{R}^d$ be equipped with the subspace σ-algebra

$$\mathcal{F}_B := \{B \cap N \mid N \in \mathcal{B}(\mathbb{R}^d)\}.$$

(2) $B = \{\sim_1 b_1, \ldots, \sim_n b_n\}$,
(3) $V = \top$,
(4) $V_1 = \{v_1, \ldots, v_{n_1}\}$ with $\mathsf{recovers}(B_1, V_1)$ and $c_1 \in V_1$ due to $\mathsf{has}(V_1, c_1)$ by I.H.
(5) $V_2 = \{w_1, \ldots, w_{n_2}\}$ with $\mathsf{recovers}(B_2, V_2)$ and $c_2 \in V_2$ due to $\mathsf{has}(V_2, c_2)$ by I.H.

First observe that we have

$$\begin{aligned}
&\mathcal{D}[\![e : \tau]\!] : \mathcal{D}[\![\tau]\!] \\
&= \mathtt{DC}_{B,B_1,B_2,V_1,V_2}(\mathcal{D}[\![c_1 : \mathbf{float}[B_1; V_1]]\!], \mathcal{D}[\![c_2 : \mathbf{float}[B_2; V_2]]\!]) : \mathbf{fin}(|B| + 1) \\
&= \mathtt{DC}_{B,B_1,B_2,V_1,V_2}(k_1 : \mathbf{fin}(|B_1| + 1), k_2 : \mathbf{fin}(|B_2| + 1)) : \mathbf{fin}(|B| + 1)\,,
\end{aligned}$$

where k_i is the unique number such that $c_i \in \mathrm{Intervals}(B_i)_{k_i}$. By Table 1 and Figure 13, this yields

$$\begin{aligned}
&[\![\mathcal{D}[\![e : \tau]\!]]\!] \\
&= [\![\mathbf{discrete}(p_0, \ldots, p_n)]\!] \text{ where } p_k = \Pr[X \in \mathrm{Intervals}(B)_k] = H(b_{k+1}) - H(b_k), \quad X \sim \mathrm{Uniform}(c_1, c_2)\,.
\end{aligned}$$

Here $c_1 < c_2$, and the uniform CDF is the clamped function

$$F_{c_1,c_2}(x) = \begin{cases} 0 & x \le c_1, \\ \dfrac{x - c_1}{c_2 - c_1} & c_1 < x < c_2, \\ 1 & x \ge c_2. \end{cases}$$

Thus each p_k is the difference of F_{c_1,c_2} at consecutive cut locations (with endpoints $-\infty$ and $+\infty$). Strict and non-strict cuts assign a boundary point to different adjacent intervals, but this does not change these masses because $c_1 < c_2$ makes the uniform distribution atomless.

For Lemma F.43.2, consider the following:

$$\begin{aligned}
&\pi_1([\![e : \tau, \mathcal{D}[\![e]\!] : \mathcal{D}[\![\tau]\!]]\!]) \\
&= [\![e : \tau, \mathcal{D}[\![e]\!] : \mathcal{D}[\![\tau]\!]]\!] \gg\!= \lambda(e_1, e_2).\delta_{e_1} && \text{(definition)} \\
&= (\mathrm{Uniform}(c_1, c_2) \gg\!= \lambda v.\delta_{(v:\mathbf{float}[B;V], v_i:\mathbf{fin}(|B|+1))}) \gg\!= \lambda(e_1, e_2).\delta_{e_1} \\
&\qquad && \text{(Table 3, } v_i \text{ is unique number s.t. } v \in \mathrm{Intervals}(B)_{v_i}) \\
&= \mathrm{Uniform}(c_1, c_2) \gg\!= \lambda v.(\delta_{(v:\mathbf{float}[B;V], v_i:\mathbf{fin}(|B|+1))} \gg\!= \lambda(e_1, e_2).\delta_{e_1}) \\
&\qquad && \text{(monad assoc., } v_i \text{ is unique number s.t. } v \in \mathrm{Intervals}(B)_{v_i}) \\
&= \mathrm{Uniform}(c_1, c_2) \gg\!= \lambda v.\delta_{v:\mathbf{float}[B;V]} \\
&= \mathrm{Uniform}(c_1, c_2) \\
&= [\![e : \tau]\!]\,. && \text{(Table 1)}
\end{aligned}$$

For Lemma F.43.2, consider the following:

$$\begin{aligned}
&\pi_2([\![e : \tau, \mathcal{D}[\![e]\!] : \mathcal{D}[\![\tau]\!]]\!]) \\
&= [\![e : \tau, \mathcal{D}[\![e]\!] : \mathcal{D}[\![\tau]\!]]\!] \gg\!= \lambda(e_1, e_2).\delta_{e_2} && \text{(definition)} \\
&= (\mathrm{Uniform}(c_1, c_2) \gg\!= \lambda v.\delta_{(v:\mathbf{float}[B;V], v_i:\mathbf{fin}(|B|+1))}) \gg\!= \lambda(e_1, e_2).\delta_{e_2} \\
&\qquad && \text{(Table 3, } v_i \text{ is unique number s.t. } v \in \mathrm{Intervals}(B)_{v_i}) \\
&= \mathrm{Uniform}(c_1, c_2) \gg\!= \lambda v.(\delta_{(v:\mathbf{float}[B;V], v_i:\mathbf{fin}(|B|+1))} \gg\!= \lambda(e_1, e_2).\delta_{e_2}) \\
&\qquad && \text{(monad assoc., } v_i \text{ is unique number s.t. } v \in \mathrm{Intervals}(B)_{v_i}) \\
&= \mathrm{Uniform}(c_1, c_2) \gg\!= \lambda v.\delta_{v_i:\mathbf{fin}(|B|+1)} && \text{(monad return, } v_i \text{ is unique number s.t. } v \in \mathrm{Intervals}(B)_{v_i}) \\
&= [\![\mathbf{discrete}(p_0, \ldots, p_n)]\!] \text{ where } p_k = \Pr[X \in \mathrm{Intervals}(B)_k] = H(b_{k+1}) - H(b_k) \\
&\qquad && \text{(pushforward along the unique interval-index map)} \\
&= [\![\mathcal{D}[\![e]\!] : \mathcal{D}[\![\tau]\!]]\!]\,. && \text{(Table 1)}
\end{aligned}$$

Now, the remaining cases are either (i) trivial base cases or (ii) immediately follow from the I.H. and Lemma F.44 further below. We demonstrate this for the case

$$e : \tau = \mathbf{uniform}(e_1 : \mathbf{float}[B_1; V_1], e_2 : \mathbf{float}[B_2; V_2]) : \mathbf{float}[B; V]\,,$$